LLMCOMPASS · REFERENCE DESIGN · 2026-10-29

面向多精度部署组合的 Tensor Core 算力配比探索

在固定 M/N 网格上，通过扩展点积单元的 K 并行度，比较 1:2:4、1:2:6、1:2:8、1:4:8。先完成可复现的计算建模，再用真实 PPA、供数约束和跨模型验证决定芯片配比。

`feature/precision-ratio-kexpansion` · FP16/BF16 · MXFP8/6 · MXFP4/NVFP4/HiF4

已实现

格式描述、分组存储字节数、三档配比、真实 SCALE-Sim OS token 调度、GEMM 接入、统一精度实验入口。

等待数据

候选 Tensor Core 的面积、各格式功耗、可实现频率；目标模型的算子轨迹、非 GEMM 时间与精度验证。

仍需扩展

局部 SRAM 时序与阻塞、格式转换、真实芯片校准、固定芯片面积/功耗评估。

研究问题：在规定的芯片预算与质量要求下，哪种多精度算力配比能在一个预先定义的模型部署组合上，获得更好的吞吐、延迟或能效，同时控制未参与设计的模型与高精度负载的退化?

1. 故事应当围绕部署组合与硬件预算展开

算力配比是硬件资源分配问题。传统位宽递减所对应的峰值增长规律，不自动等于现代推理部署的最佳资源分配。提出更高的低精度算力后，需要证明新增面积和功耗在多种负载上值得投入。

只保持核数与频率不变，再观察 FP4 GEMM 变快，会得到较容易预期的结果。能够支持芯片决策的结论应进一步比较：同样的额外面积，用于低精度 K 扩展、增加常规计算引擎、增加操作数带宽，哪一种在目标部署组合中更有价值。

通用性来自通用 GEMM 接口、保留高精度功能、一个冻结的硬件选择，以及跨模型家族和运行条件的验证。无需要求模型内部混合使用三种精度；单个模型的 GEMM 可以统一使用一种格式，整个部署组合仍可能同时包含 BF16、MXFP8 与 FP4 模型。

需要接受的边界：增加 FP4 算力无法直接加速统一 BF16 GEMM；保持核数时它可能没有性能变化，固定芯片面积时还可能因核数减少而变慢。应当量化这种代价，不能用"所有模型都会受益"作为研究前提。

2. 统一 GEMM 精度与混合精度的区别

本轮的"统一精度"指一份负载中的 GEMM 都使用同一种 A/B 格式，累加使用 FP32。Softmax、归一化、路由、通信等可以保持它们需要的格式与测量时间。输入实验入口的非 GEMM 时间固定，不随低精度峰值自动缩短。

比较	固定项	改变项	能归因于什么
配比实验	模型、格式、精度校验、batch、上下文、核数与频率	1:2:4 → 1:2:8 等	计算资源分配及其实现代价
格式实验	模型、质量门槛、硬件	BF16 → MXFP8 / NVFP4 等	量化、存储和计算格式的共同影响
预算实验	总芯片面积或功耗、HBM/IO 条件、质量门槛	配比、核数、必要的存储组织	芯片设计选择的实际价值

同一配比实验中，不能把基准 FP4 与新设计 FP4 的权重存储量再缩小一次。两者已有相同数据格式，新增压缩收益为零。W4A16 解量化到 FP16 的路径也不能直接记为原生 FP4 GEMM。

3. 格式与三档计算能力的表示

格式	payload	本参考设计的计算档位	默认存储描述
FP16 / BF16	16 bit	16 bit	无共享 scale
MXFP8	8 bit	8/6 bit	32 元素 + 8 bit scale
MXFP6	6 bit	8/6 bit	32 元素 + 8 bit scale；紧凑 6-bit packing
MXFP4	4 bit	4 bit	32 元素 + 8 bit scale
NVFP4	4 bit	4 bit	16 元素 + 8 bit scale，另加每 tensor 的 FP32 scale
HiF4	4 bit	4 bit	64 元素 + 32 bit 分层元数据

MXFP6 的不同编码保留名称，但在该参考设计中共享计算档位。三种 4-bit 格式也共享配置的峰值；这是一项候选硬件能力假设，不能据此认为现有任意 GPU/NPU 都原生支持所有格式，或它们的数值误差、功耗和访存相同。

存储计数会按分组轴补齐整个 block。A 的分组沿 K；数学形状为 [K,N] 的 B 也沿 K 分组，因此使用第二倒数轴。它们不能都机械地按最后一维分组。宏 word 对齐、swizzle、复本和 scale 重读需要由后续局部存储后端补充。

格式依据：CUTLASS scaled operands、PTX block-scaled MMA、NVIDIA NVFP4 说明、HiF4 原始论文。这里实现描述和成本计数，没有实现数值量化器或低精度 GPU 算子。

4. K 扩展如何进入 SCALE-Sim

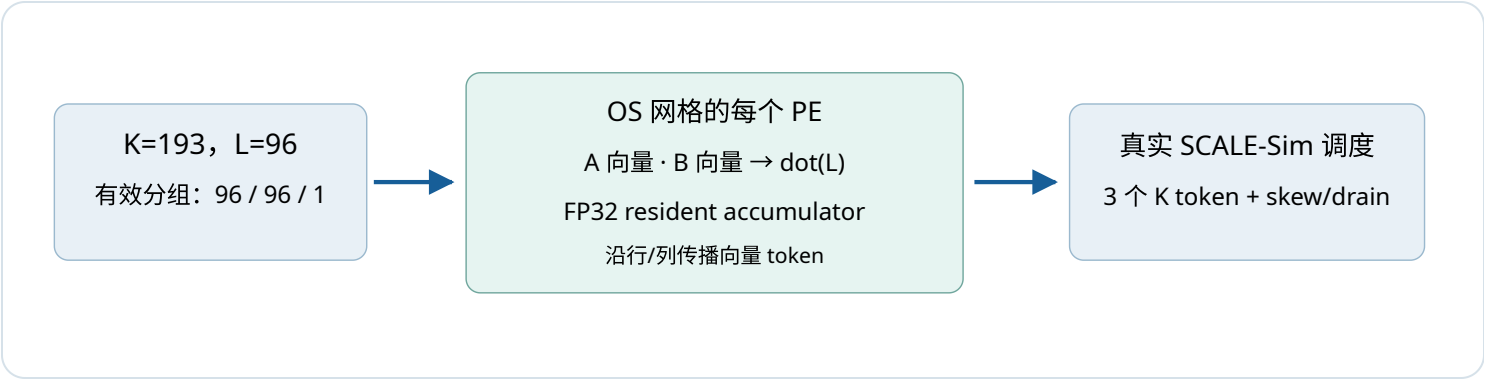
配比	16-bit L	8/6-bit L	4-bit L	M/N 网格
1:2:4	16	32	64	固定 R×C
1:2:6	16	32	96	固定 R×C

配比	16-bit L	8/6-bit L	4-bit L	M/N 网格
1:2:8	16	32	128	固定 R×C
1:4:8	16	64	128	固定 R×C

对于 D=A×B，每个 PE 的一次更新计算 L 个标量乘积并更新 FP32 部分和。将原 K 转成 Q=ceil(K/L) 个向量 token；SCALE-Sim 的 OS 单元对每个 token 执行一次点积更新。A/B 的 token 分别沿行/列传播，保留原有 skew 和排空时隙。

```
原问题：M × N × K
点积模式：R × C 个 PE，每个 PE 每个阵列 step 处理 L 个乘积
SCALE-Sim 计算拓扑：M_tile × N_tile × ceil(K/L)
峰值：R × C × L MAC/step
最后一组：只记录真实 K 的有效 MAC，其余为尾部空槽
```

这是对点积阵列的一种计算抽象：它要求点积结果/累加更新以及 PE 链路能按一个 step 工作，操作数和 scale 已就绪。L 从 64 增大到 96/128 后能否保持相同频率和更新速率，需要综合或实测数据支撑。当前没有将更长归约树的代价凭空设为已验证事实。



当前后端只支持该 OS 映射。WS/IS 会改变空间维度和驻留位置，不能简单把 K 压缩后沿用同一解释；代码对此显式报错。SCALE-Sim 使用已安装、已验证的 2.0.2 计算组件，没有修改第三方源码，也没有把依赖升级为 v3。

5. 周期、尾块与多引擎调度

一个任务覆盖至多 R×C 的输出，沿完整 K 归约。多个独立任务按波次分配到芯片上的计算引擎；批量 GEMM 保持独立输出，不能拼接 K 后假装得到同一个问题。

读取 SCALE-Sim demand matrix 的行数作为包含首拍的已分配时隙，不把最后一个从零开始的 timestamp 当作周期数。对于该版本、一个输出 tile，分配时隙为 Q+R+C−2；真实结果由组件生成，并不直接使用这个公式代替调用。

单个 16×16 网格，FP4	E64	E128	含义
K=4096	64+30=94 拍	32+30=62 拍	1.516×，填充/排空没有减半
K=16	1+30=31 拍	1+30=31 拍	没有收益；新增点积槽无法被填满

单个 16×16 网格，FP4	E64	E128	含义
K=193，E96	3 组、最后 1 个有效元素；K packing=193/288		保留真实 MAC 数，不把 padding 算成有用工作

利用率分别报告 K packing、M/N 空间填充、波次填充和流水线效率。整机有效利用率为有效 MAC/(所有引擎峰值×计算周期)，避免把只观察忙引擎得到的高利用率当作整个芯片的利用率。

6. 已接入的代码与调用方式

格式
`software_model/utils.py`
计算档位、scale、分组、storage_bytes

硬件
`hardware_model/precision.py`
基础 L 与三档倍率

周期
`systolic_array_model/precision_k.py`
SCALE-Sim token 调度、波次与利用率

算子
`software_model/matmul.py`
普通/批量 GEMM，M=1 也进入新后端

实验
`scripts/study_precision_ratios.py`
统一格式扫描、PPA、留出集统计

```
from hardware_model.precision import KExpansionProfile
from hardware_model.compute_module import SystolicArray
from software_model.utils import data_type_dict, Tensor
from software_model.matmul import Matmul

array = SystolicArray(16, 16, 16, 2, 4,
    precision_profile=KExpansionProfile(16, (1, 2, 6)))
# 将 array 放入自己的 Core / ComputeModule / Device。
dtype = data_type_dict["mxfp4"]
gemm = Matmul(dtype, output_type=data_type_dict["bf16"])
gemm(Tensor([128, 4096], dtype), Tensor([4096, 4096], dtype))
seconds = gemm.compile_and_simulate(device, "precision-k")
details = gemm.precision_result
```

新硬件必须显式设置 precision_profile；未设置的历史配置保留旧路径。启用后，三档共享同一阵列，旧 mac_per_cycle 不再作为第二次倍率应用。JSON 硬件适配器也支持在 systolic_array 下增加 precision_profile。

```
"precision_profile": {
  "base_dot_length": 16,
  "ratios": [1, 2, 8]
}
```

旧按 FPU 估算阵列面积的函数遇到新 profile 会拒绝计算，避免遗漏低精度新增硬件却仍报告旧面积。精度 profile 的面积使用外部 PPA 表。

7. SRAM：本轮的精确范围与下一步衔接

本轮输出的是计算时间线和向量 token 数，token 既不是 SRAM 地址，也不是一次完整 SRAM word 事务。尚未由源数据、广播寄存器、collector、bank 和端口仲裁还原实际供数，所以不会声称已经得到真实 L0 每拍读取或硬件可行性。

算子入口还给出 max(计算调度时间，HBM/L2 必需传输时间) 的理想重叠估计。必需字节计入 A/B 数据和元数据、D 写回；默认 β=0，不读初始 C。它没有计入跨 tile 重复搬运、局部供数、转换或量化计算，不是实测 kernel 延迟。

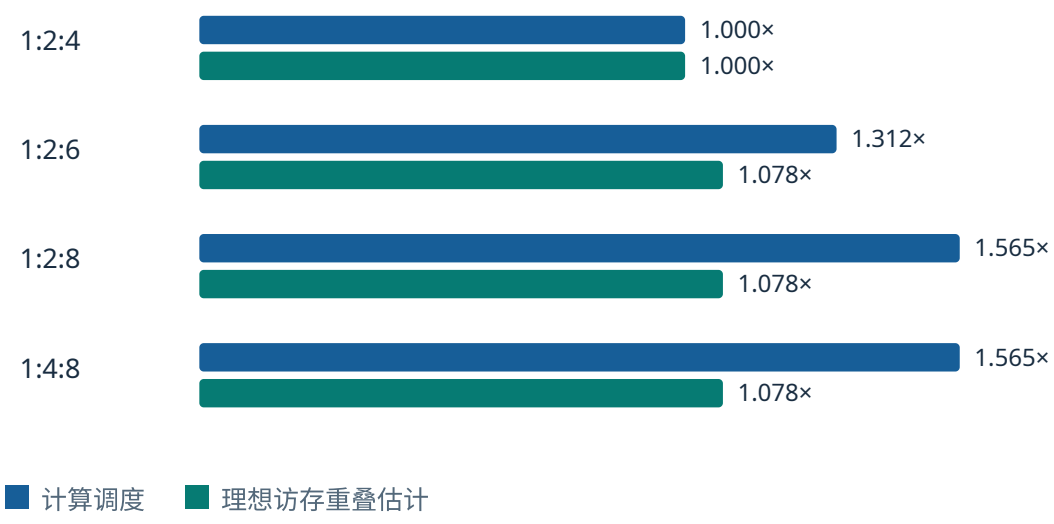
下一阶段在同一 profile 下接入：操作数载入与驻留规则 → 每拍请求 → bank/端口仲裁 → 数据就绪 → 阵列推进。复用寄存器能否摊薄 SRAM 带宽，需要通过加载截止时间、寄存器容量与缓冲槽判断。

存储实验	保持/改变	必须报告
现有 SRAM	容量、bank、lane、端口都保持	新增计算后产生多少阻塞；原带宽余量能支撑哪些模式
联合升级	在固定总容量下搜索宏组织	为达到供数要求增加多少面积、动态能耗、泄漏和互连成本
完整预算	把新增 TC+SRAM+RF+控制放回芯片预算	核数/频率是否改变，以及全芯片结果

已有 `cost_model/cacti.py`、`local_sram.py` 可复用宏成本查询；当前 `dot_product_sram.py` 的结构化访问假设不能自动视为本后端的已验证物理访存。两者尚未闭环。

8. 已跑通的示例：计算收益不等于负载收益

下面数据来自本分支实际调用 SCALE-Sim 的扫描：4 组示例形状、7 种格式、4 个配比，共 112 组。示例是人为配置的形状代理，非模型实测；非 GEMM 时间为零。蓝色是计算调度加速比，绿色加入必需访存与理想重叠估计。两者都以相同格式的 1:2:4 为基准。



配比	计算加速比	重叠估计加速比	计算时间 / μs	重叠估计时间 / μs	TC 面积比
1:2:4	1.000×	1.000×	106.320	108.526	1.000
1:2:6	1.312×	1.078×	81.048	100.688	待提供
1:2:8	1.565×	1.078×	67.920	100.688	待提供
1:4:8	1.565×	1.078×	67.920	100.688	待提供

没有填写候选 PPA 时，面积显示“待提供”；不会根据精度位宽或吞吐倍率生成看似真实的面积/功耗。蓝绿之间的差距仅用于演示瓶颈分析方法。

9. 如何提供面积和功耗

模板为 `configs/precision_ppa_template.json`。一份文件对应同工艺、电压、频率条件下的一组候选设计。面积表示一个完整共享多精度 Tensor Core 相对基准的面积，不能把三个模式面积相加。

```
{
  "scope": "one_tensor_core",
  "process_nm": 22,
  "voltage_v": null,
  "frequency_hz": 1000000000,
  "profiles": {
    "1:2:4": {"area_relative": 1.0,
      "active_power_relative": {"16": 1.0, "8": 1.0, "4": 1.0}},
    "1:2:8": {"area_relative": null,
      "active_power_relative": {"16": null, "8": null, "4": null},
      "active_power_relative_by_format": {"hif4": null}}
  }
}
```

每种格式的功耗比应相对该格式的基准设计；可用 per-format 字段覆盖共享档位。当前能输出 TC 面积归一化吞吐比，以及按提供的活动功耗×分配给任务的阵列周期计算的活动能量代理。后者没有空闲泄漏、存储和全芯片能耗，不能直接称为 tokens/J。

12nm 和 22nm 各自建立一组表即可开展同工艺相对比较。跨工艺需要单独校准；当前 CACTI 适配没有已验证的 12nm 技术模型。更长点积的归约树、scale 支持、FP32 累加反馈和互连必须包含在 PPA 的明确统计范围中。

10. 避免专用设计：预先冻结的实验矩阵

维度	建议覆盖	要防止的偏差
模型结构	Dense GQA、不同 Attention 结构、MoE；同家族不同规模	仅换同一模型的 hidden size 不能证明跨家族通用性
运行阶段	Prefill、decode、批处理 decode、专家小 batch	只挑大 GEMM，遗漏内存受限和利用率低的场景
长度/batch	短/长上下文，batch=1 与服务目标范围内的 batch	用大 batch 的收益代表低延迟部署
格式	BF16 控制组、MXFP8/6、至少两种 4-bit 格式；每次配比比较固定格式	把量化收益混进硬件收益，或忽略元数据差异
形状	长/短 K，M 很小、N 很大，K=96/128 边界和尾数，真实非对齐维度	只选被新点积长度整除的尺寸
质量	每个模型/格式独立量化与质量评估；固定可接受门槛	性能模拟器不能证明所有算子都可无损转成 FP4
选型/验证	按模型家族分组，预留整个家族和未见 batch/长度；选型后冻结设计	根据测试集反复选择配比，造成数据泄漏

当前配置中的 design/holdout 字段与统计入口已经实现，但示例留出集仍是形状代理。正式研究需要替换为真实模型轨迹，记录框架、kernel、量化配置、形状分布和哈希；不能把 112 组示例描述成 112 个真实模型实验。

部署组合的权重应来自事先给定的部署目标或明确的等权规则。建议同时报告按工作负载等权的几何平均、最差加速比、退化比例、各模型/阶段明细，并补充不同合理权重下的敏感性。不要让一个有大量重复形状的模型在统计中自动获得更高权重。

11. 核心对照与论文图表

实验/图	输入与对照	支撑的结论
E1：形状收益图	固定 M/N 网格，扫 K、M、batch、四种配比；包含尾块	收益来自哪些 K 并行度与空间利用率条件，何时饱和
E2：逐算子分解	相同模型与格式下比较各 GEMM 和非 GEMM 时间	端到端收益来自何处；Amdahl 与瓶颈迁移
E3：存储消融	现有 SRAM / 联合升级 / 理想无限供数三组	计算上限与真实供数之间的差距，以及存储升级价值
E4：等预算竞争	低精度 K 扩展 / 增加传统 1:2:4 计算 / 增加局部带宽	额外资源为什么值得用于改变配比
E5：跨家族留出	只在设计集选一个硬件方案，原样运行留出集	收益是否迁移；报告不受益/退化的模型和原因
E6：鲁棒性	变动部署权重、SRAM 有效效率、频率、PPA 误差	最佳配比是否稳定，收益有无超过建模不确定性
E7：质量约束	独立验证各模型的固定精度策略	结果对应实际可部署的低精度负载

可在设计集选择：最大化加权对数吞吐收益，同时限制面积、功耗和高精度/尾部场景退化。选择规则与退化阈值必须在看留出结果之前确定。本轮脚本提供分组统计，没有自动使用留出结果推荐最优芯片。

12. 固定芯片预算下的公平比较

```
A_core(design) = A_other_core + n_TC × A_TC(design)
                 + A_operand_storage(design) + A_added_interconnect(design)
N_core(design) = floor((A_die_budget - A_fixed_IO) / A_core(design))
```

上式只是预算组织方式：必须使用互不重复的面积范围，原 core 中已包含的 SRAM/RF 不应再次计费。减少核数后，向量、寄存器、局部 SRAM、调度等资源也可能一起减少；不能只把 TC 数乘一个面积比例而保留其他核心资源不变。

固定功耗实验需要活动率、泄漏、存储/互连、频率电压等信息。当前提供的 TC 相对功耗只覆盖其中一部分。应先报告固定核数的增量成本，再补全全芯片成本，避免把 TC 面积效率当作全芯片效率。

一个更完整的对照是：给传统配比相同的新增面积，允许它选择增加计算或带宽；然后比较统一低精度模型和 BF16 控制负载。若 1:2:8 仅在理想供数且不计面积的条件下领先，应该将其判为条件性收益。

13. 复现命令与接入现有 Transformer

```
# 四配比 × 七格式 × 四代理负载；原生 token trace 可选
.venv/bin/python scripts/study_precision_ratios.py \
  --config configs/precision_ratio_study.json \
  --ppa configs/precision_ppa_template.json \
  --output-dir artifacts/precision-ratios/my-run --trace

# 从仓库现有 dense Transformer block 导出形状；不是现代模型性能实测
.venv/bin/python scripts/export_transformer_gemms.py \
  --hidden 4096 --heads 32 --batch 1 --sequence 512 --phase prefill \
  --output artifacts/precision-ratios/legacy-prefill.json
.venv/bin/python scripts/study_precision_ratios.py \
  --config artifacts/precision-ratios/legacy-prefill.json \
  --output-dir artifacts/precision-ratios/legacy-prefill

.venv/bin/python -m unittest discover -s tests -v
```

输出 `result.json` 包含逐算子调度、存储计数、利用率、源文件哈希、SCALE-Sim 版本及计算组件哈希；`summary.csv` 便于画图。可选 NPZ 是原生向量 token demand matrix，不能当作 SRAM trace。输出目录必须为空，以保留既有结果。

真实模型接入时，将 workload.ops 换为实际 GEMM 列表；count 表示重复执行次数，batch_count 表示独立批量矩阵，不等于请求 batch。non_gemm_seconds 表示这一整份 workload 的其他时间，需与重复层数和运行阶段保持同一统计范围。

14. 验证与实施顺序

阶段	当前状态/下一步	验证条件
P0：格式与计算	已实现。格式、尾块、周期、波次、普通/批量 GEMM、JSON 接入。	测试重放真实 token 的行/列传播，用整数向量点积与 NumPy GEMM 对照；不宣称浮点误差验证。
P0：历史回归	运行原有 CPU 基线，保留旧配置、旧查表与标量后端。	9 项基线与原始 CSV 哈希保持；独立验证新路径不会误用旧面积。
P1：存储闭环	增加已定义微架构的加载、驻留、scale、端口/银行调度，并接入宏成本。	小阵列逐拍请求/服务轨迹；尾部、缓冲翻转、冲突与阻塞。
P1：目标芯片校准	用现有 H100/B200/B300 与 Ascend 平台验证对应原生模式。	计算饱和、带宽饱和、小形状、访存计数的分离微基准；按形状保留验证集。
P2：设计研究	接入真实 PPA 和真实模型；冻结组合权重及候选选择，再运行留出集。	等预算对照、退化约束、Pareto 与敏感性；自定义配比需综合/RTL/单元数据支撑。

现有芯片只能校准其实际支持的模式和对应层次，无法直接证明未制造的 1:2:6 设计。不要用一个全局拟合因子补偿所有形状与格式；应分开校准计算更新、填充排空、存储、转换和启动，并检查设计排序是否在误差范围内稳定。

本报告对应一个可运行的参考计算模型和研究实施方案。相对 PPA 待输入；示例结果不是量化精度验证、GPU/NPU 性能实测或全芯片最优配比结论。